

AI-ASSISTED BACKEND ENGINEERING

Text classification, built for integration.

A production-ready MERN service that classifies customer messages as Complaint, Query, Feedback, or Other and returns a validated confidence score.

Solution overview

The application combines a focused REST API, local LLM inference through Ollama, optional MongoDB persistence, robust output validation, and a responsive React interface for demonstration.

4

Supported categories

1

Primary POST endpoint

3/3

Automated tests passing

GITHUB

github.com/sandipan-das-sd/AI-Assisted-Text-Classification-API

LIVE UI

ai-assisted-text-classification-api.vercel.app

API HEALTH

ai-assisted-text-classification-api-delta.vercel.app/api/health

From message to meaningful category.

The service keeps routing, request handling, model communication, validation, and persistence separated for maintainability and testing.

1

React client

Collects customer text and sends a JSON request.

2

Express API

Validates the request and delegates classification.

3

Ollama LLM

Returns a category and confidence in strict JSON.

4

MongoDB

Optionally stores successful results for history.

API contract

```
POST /api/classify
Content-Type: application/json

{
  "text": "My order arrived damaged."
}

{
  "category": "Complaint",
  "confidence": 0.96
}
```

Classification definitions

- **Complaint:** dissatisfaction or a reported problem.
- **Query:** a question or request for information.
- **Feedback:** an opinion, suggestion, praise, or review.
- **Other:** text outside the defined categories.

Reliability safeguards

- Rejects empty input and text over 5,000 characters.
- Validates AI categories against an allow-list.
- Clamps confidence between 0 and 1.
- Uses a deterministic temperature of 0.
- Returns consistent JSON errors.

Technology stack

React

Vite

Node.js

Express

MongoDB

Mongoose

Ollama

LangChain

Vercel

The API also works without MongoDB; classification continues while persistence is skipped.

Implementation highlights.

The submission covers the requested endpoint, layered backend structure, minimal error handling, AI integration, documentation, and optional Postman support.

**Single POST endpoint**

`/api/classify` accepts a text string.

**AI classification**

Ollama runs the `llama3.2:3b` model.

**Structured response**

Category and confidence are normalized.

**Layered backend**

Routes, controllers, services, and models.

**Error handling**

400, 404, and 500 JSON responses.

**Documentation**

README includes setup and API examples.

**Postman collection**

Ready-to-run health and classify requests.

**Automated tests**

Three API/service tests are passing.

Run locally

```
ollama pull llama3.2:3b

cd server
npm install
copy .env.example .env
npm start

cd client
npm install
npm run dev
```

Repository contents

- Complete frontend and backend source
- Environment variable examples
- Automated API tests
- Deployment configuration
- Postman collection
- Detailed README documentation

Deployment note: The demonstration uses a temporary secure tunnel to reach local Ollama. For a permanent production environment, the model should run behind an authenticated cloud endpoint or be replaced with a managed AI provider.